SPECFIX – AI-Driven Requirement

Ambiguity Detection and Test Specification System

A Phase 1 (60%) Progress Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering
Specialization in
Artificial Intelligence and Machine Learning

By

Shrikant Sharma (2415500449)
Raghav Bansal (2415500377)
Udit Bhatt (2415500484)

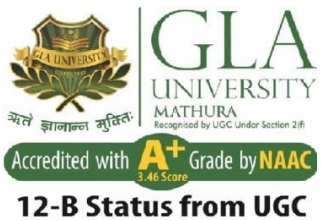
Group No. 138

Under the Guidance of

Mr. Preshit Mangesh Desai

Department of Computer Engineering & Applications

Institute of Engineering & Technology



GLA University
Mathura- 281406, INDIA

October, 2026

DECLARATION

We hereby declare that the work which is being presented in the B.Tech. Project **"SpecFix – AI-Driven Requirement Ambiguity Detection and Test Specification System"**, in partial fulfilment of the requirements for the award of the Bachelor of Technology in Computer Science and Engineering Specialization in Artificial Intelligence and Machine Learning and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of our own work carried out under the supervision of Mr. Preshit Mangesh Desai, Technical Trainer.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign:
Name of Candidate: Shrikant Sharma
Univ. Roll No.: 2415500449
Course: B.Tech. CSE (AIML)

Sign:
Name of Candidate: Raghav Bansal
Univ. Roll No.: 2415500377
Course: B.Tech. CSE (AIML)

Sign:
Name of Candidate: Udit Bhatt
Univ. Roll No.: 2415500484
Course: B.Tech. CSE (AIML)

CERTIFICATE

This is to certify that the above statements made by the candidates are correct to the best of my/our knowledge and belief.

Supervisor
(Mr. Preshit Mangesh Desai)
Technical Trainer
Dept. of Computer Engg. & App.

Project Co-ordinator
(Dr. Mayank Srivastava)
Associate Professor
Dept. of Computer Engg. & App.

Program Co-ordinator
(Dr. Nikhil Govil)
Associate Professor
Dept. of Computer Engg. & App.

Date: 21 October, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout this project. First and foremost, we extend our heartfelt thanks to our respected guide, Mr. Preshit Mangesh Desai (Technical Trainer), for his continuous encouragement, valuable feedback and insightful suggestions at every stage of the project titled **"SpecFix – AI-Driven Requirement Ambiguity Detection and Test Specification System"**. His guidance has been fundamental in shaping both the direction and the discipline of this work.

We are deeply thankful to the Department of Computer Engineering and Applications and to our institution for providing a supportive learning environment, access to essential resources and the facilities that enabled the smooth execution of this project. Building SpecFix has significantly strengthened our knowledge of large language models, database design, web engineering and software testing, as well as our teamwork and problem-solving skills.

We would also like to thank the Head of the Department, Mr. Ashok Bhansali, for his constant support and motivation. We are grateful to all faculty members of the department for their guidance, and to our classmates and friends for their cooperation and suggestions. Lastly, we owe our deepest appreciation to our families, who have been a continuous source of strength, patience and motivation.

Shrikant Sharma
2415500449

Raghav Bansal
2415500377

Udit Bhatt
2415500484

ABSTRACT

Software requirements are usually written as short tickets or user stories in tools such as Jira. These tickets are frequently ambiguous or incomplete: a word such as “quickly” hides a decision, an acceptance criterion cannot be tested, or an edge case is never mentioned. Developers fill the gaps with assumptions, and the mismatch with business intent is discovered late — in QA or after release — when it is most expensive to fix.

SpecFix is an AI-assisted requirements-quality platform that catches these gaps before development starts. A ticket goes in; structured *flags* come out, each with a category (missing information, vague language, contradiction, unhandled edge case, security/compliance gap or untestable criterion), the exact text it refers to, why it matters, and a question for the product manager. An *Extractor* language-model call proposes candidate flags and an *Adversarial Critic* call removes false alarms. A human reviewer then accepts, edits or dismisses every flag in a web dashboard, with an append-only audit trail and no automatic resolution path.

Once a ticket is approved, the *Spec Compiler* turns its acceptance criteria into Gherkin scenarios, and a deterministic generator turns each approved scenario into a Playwright test scaffold with marked placeholder selectors. A traceability graph backed by pgvector embeddings shows clause-level coverage and detects scenarios that drift when a ticket changes.

The system is a TypeScript monorepo (Next.js 15, Supabase PostgreSQL with Row-Level Security and pgvector, Azure OpenAI / OpenAI, zod, Vitest). Every model output is schema-validated, ticket text is always treated as data, every model call is logged with its cost, and the Critic’s precision is only reported next to a random-pruning control. On an 11-ticket gold set the Spec Compiler covered 91.7% of acceptance-criteria clauses, made one unsupported coverage claim in 42 scenarios (2.4%), and produced 42 of 42 valid scaffolds at about US$0.0004 per ticket.

TABLE OF CONTENTS

Declaration ii
Certificate iii
Acknowledgement iv
Abstract v
Table of Contents vi
List of Tables vii
List of Figures vii

CHAPTER 1: INTRODUCTION 1
1.1 Overview and Motivation 1
1.2 Objectives 2
1.3 Summary of Similar Applications 3
1.4 Organization of the Project Report 3

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS 3
2.1 Introduction to Requirement Analysis 4
2.2 Feasibility Study 4
2.3 Functional Requirements 5
2.4 Non-Functional Requirements 5
2.5 Use Case Descriptions 6
2.6 Module Descriptions 6

CHAPTER 3: SOFTWARE DESIGN 7
3.1 System Architecture Overview 7
3.2 Data Flow Diagrams 7
3.3 UML Diagrams 9
3.4 Database Design 12

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE 13
4.1 Technology Stack Used 13
4.2 AI Architecture 13
4.3 Frontend Implementation 15

CHAPTER 5: SOFTWARE TESTING 18
5.1 Testing Approach 18
5.2 Unit Testing 18
5.3 Test Case Specifications 18
5.4 Multi-Tenant Isolation Testing 19
5.5 Evaluation Results 19

CHAPTER 6: CONCLUSION 20
6.1 Current Status & Future Work 20

CHAPTER 7: SUMMARY 21
7.1 Summary 21

REFERENCES 22

APPENDIX A: ABBREVIATIONS 23

APPENDIX B: SAMPLE COMMANDS 23

LIST OF TABLES

Table 1.1	Comparison with Existing Tools and Approaches	3
Table 2.1	Feasibility Study Summary	4
Table 2.2	Functional Requirements of SpecFix	5
Table 2.3	Non-Functional Requirements of SpecFix	5
Table 2.4	Main Use Cases	6
Table 2.5	Modules of SpecFix	6
Table 3.1	Principal Database Tables	12
Table 4.1	Technology Stack Summary	13
Table 5.1	Unit Testing Summary by Package (offline run)	18
Table 5.2	Black Box and White Box Test Cases	18
Table 5.3	Generation Evaluation Results (gold-v1)	19
Table 6.1	Project Status by Milestone	20

LIST OF FIGURES

Figure 1.1	System Architecture of SpecFix	2
Figure 3.1	Level 0 DFD – Context Diagram	8
Figure 3.2	Level 1 DFD – Main Data Flow	8
Figure 3.3	Use Case Diagram of SpecFix	9
Figure 3.4	Class Diagram of the Core Domain	10
Figure 3.5	Sequence Diagram – Extractor and Critic Analysis	11
Figure 3.6	Entity Relationship Diagram	12
Figure 4.1	Two-Pass Analysis Pipeline and Human Review Gate	14
Figure 4.2	Spec Compiler Pipeline with Two Human Gates	15
Figure 4.3	Review Queue Page	16
Figure 4.4	Ticket Review Page with Flag Cards	16
Figure 4.5	Specs Tab: Scenario Review, Coverage and Scaffold	17